

Anchor First, Accelerate Next: Revolutionizing GNNs with PIM by Harnessing Stationary Data

Jiaxian Chen¹, Yuxuan Qi¹, Yongbiao Zhu¹, Jianan Yuan¹, Kaoyi Sun¹, Tianyu Wang¹, Chenlin Ma¹, Yi Wang^{1,2}

1. College of Computer Science and Software Engineering, Shenzhen University

2. State Key Laboratory of Radio Frequency Heterogeneous Integration

Abstract—Substantial data movement caused by irregular graph topologies hinders the efficient processing of graph neural networks (GNNs). Although the emerging near-bank processing-in-memory (PIM) architecture offers a promising solution to reduce data transfer between memory and computing units, cross-bank communication remains a critical challenge, limiting the benefits of PIM architectures. Our findings indicate that only 35.6% of the data can stay stationary within PIM units on average, with the rest requiring movement due to graph dependencies. This situation worsens as the number of PIM units increases, reducing the ratio to 18.7%.

In this paper, we argue that to fully leverage PIM architectures, systems must *maximize stationary data and minimize the movement of non-stationary data*. Following this principle, we propose *Anchor*, a scalable PIM architecture that exploits stationary data for GNNs through a hardware-software co-design approach. To maximize stationary data, we introduce the graph partitioning algorithm *Mastav*, which carefully allocates vertices and edges to preserve data locality. To minimize the movement of non-stationary data, we employ a two-step strategy. First, a customized dataflow ensures that non-stationary data is accessed and distributed exactly once. Second, an optimized communication mechanism reduces redundant data transfers through critical paths. Our extensive experiments demonstrate that *Anchor* significantly reduces processing latency and data movement compared to representative schemes.

Index Terms—GNN, Processing-in-Memory, Stationary Data

I. INTRODUCTION

Graph neural networks (GNNs) have emerged as a powerful learning paradigm for graph-structured data [11], [14], [23]. Computational efficiency is a primary concern in such deep learning systems, where vast numbers of nodes and edges have to be simultaneously processed. GNNs mainly consist of two time-consuming phases: (i) *combination* and (ii) *aggregation*. The sparse and uncertain topology of the input graph results in significant data movement, posing a severe challenge to the efficient processing of GNNs. The emerging 3D-stacked processing-in-memory (PIM) architecture [15], [22] allows computing units to be integrated near memory banks. The PIM architecture has proven to be an effective approach for addressing the data movement issue in GNNs [6], [7]. While PIM-based accelerators are already in place, the data movement problem still persists, leading to performance degradation. We observe that an unsolved key challenge is how to minimize unnecessary data movement across different PIM units.

Fig. 1 illustrates two types of vertices: *stationary* and *non-stationary*. Stationary vertices stay within their local

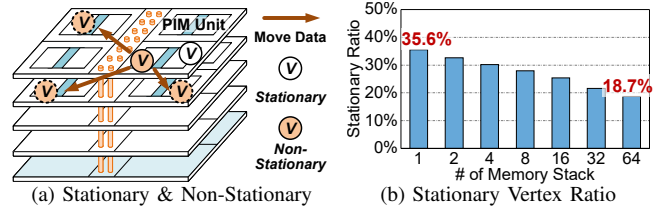


Fig. 1: Low stationary vertex ratio in PIM architectures.

processing units (PIM units) during aggregation, while non-stationary vertices have to migrate to remote PIM units due to graph dependencies. From the preliminary test results, only 35.6% of vertices are stationary vertices. Over 60% of the vertices are migrated to other PIM units. As the number of PIM units increases, the amount of stationary data drops quickly, indicating significant opportunities for optimization. To address this issue, we propose the **Max-Min principle**: *a PIM system must maximize stationary data and minimize the movement of non-stationary data*.

Many domain-specific architectures [20], [25], [30] have been proposed to address the challenges faced by GNNs. Some works capture the memory access patterns of GNNs and alleviate irregular data access by employing dedicated buffers [12], [16], [24]. Other works focus on reducing redundant memory accesses through algorithmic and architecture co-optimization [5], [10], [29]. Existing works only partially solve the issue by reducing memory accesses. Considerable efforts on architectural and system-software support are required to reduce the significant data movement.

This paper presents *Anchor*, a scalable PIM architecture harnessing stationary data to efficiently process GNNs. *Anchor* follows the proposed Max-Min principle using a hardware-software co-design approach. The objective is to fully exploit the PIM architecture and reduce data movement. At the software level, we introduce the graph partitioning algorithm *Mastav*, which maximizes stationary data by strategically allocating vertices and edges to maintain data locality. At the hardware level, we design a customized PIM architecture with a two-step optimization to minimize the movement of non-stationary data. A hybrid dataflow ensures that each non-stationary vertex is distributed only once, while an optimized communication mechanism further minimizes redundant data transfers through critical paths. We evaluate the proposed *Anchor* using a set of extensive experiments. The experimental results show that *Anchor* can significantly reduce latency and

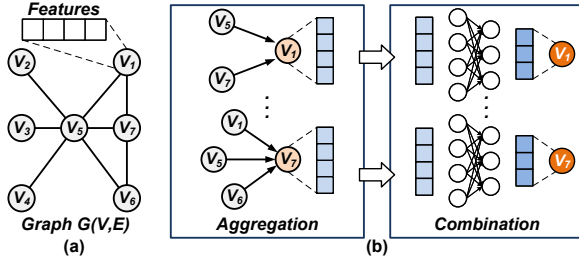


Fig. 2: (a) An input graph. (b) Illustration of a GNN model.

data movement compared with representative schemes.

The main contributions of this work include:

- *Principle*: we propose the Max-Min principle to maximize stationary data and minimize the movement of non-stationary data.
- *Algorithm*: A novel graph partitioning algorithm *Mastav* is proposed to maximize stationary data within PIM units.
- *Dataflow*: A hybrid dataflow and an optimized communication mechanism are introduced to reduce redundant data movement.
- *Architecture*: A scalable PIM architecture with dedicated compute and control logic is carefully designed to implement the Max-Min principle.

II. BACKGROUND

A. Graph Neural Networks (GNNs)

A GNN model generally consists of multiple GNN layers [11], [14], each with two main phases: aggregation and combination. As shown in Fig. 2(a), GNN models take a graph $G(V, E)$ as input, where V and E denote vertices and edges, respectively, and each vertex is represented by a feature vector. In the aggregation phase, each vertex aggregates features from neighboring vertices using an *aggregate* function, such as the mean function. In the combination phase, each aggregated feature vector is further transformed via a *combine* function, typically implemented as a multi-layer perceptron (MLP). Fig. 2(b) illustrates a typical GNN inference workflow. Vertex V_1 aggregates features from neighbors V_5 and V_7 , generating a four-dimension feature vector. An MLP then transforms it into three dimensions. The k -th GNN layer is defined as:

$$X^{k+1} = \sigma(AX^k W^k) \quad (1)$$

Here, σ denotes the non-linear activation function, A is the sparse matrix derived from E , and X^k and W^k represent the feature and weight matrices for the k -th layer, respectively. The execution order of the combination and aggregation phases can be interchanged, and we adopt a combination-first approach to reduce computations, as suggested in prior work [9], [16].

B. Graph Partitioning for GNNs

Source-cut and mixed-cut are two typical edge partitioning methods. Source-cut partitioning [26] assigns a vertex and all its incoming edges to the same partition, making it suited for GNNs since they aggregate vertices using incoming edges. Fig. 3(b) presents an example for the source-cut partitioning. During the aggregation phase, V_5 pulls remote vertices V_1 ,

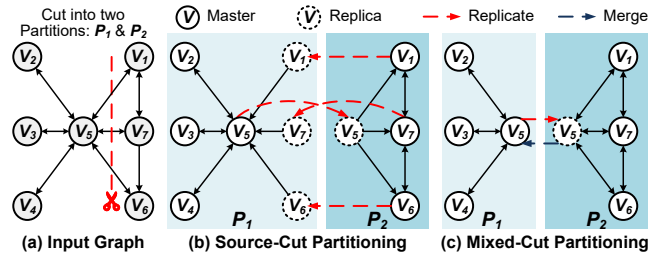


Fig. 3: Source-cut partitioning and mixed-cut partitioning.

V_6 , and V_7 from P_2 , creating their replicas in P_1 . This allows V_5 to perform aggregation operations locally in P_1 . Similarly, V_7 pulls V_5 from P_1 to P_2 . Mixed-cut partitioning [27] allocates edges to the vertex with a lower degree, allowing low-degree vertices to remain stationary and reducing replicas. This approach requires an additional *merging* phase to gather updated replicas since incoming edges of a vertex may span multiple partitions. As shown in Fig. 3(c), V_5 creates a replica in P_2 , with edges (V_1, V_5) , (V_6, V_5) , and (V_7, V_5) assigned to vertices in P_2 due to their lower degrees. Partial aggregation of V_5 occurs in P_2 , and the replica is merged back to P_1 . Mixed-cut partitioning requires only two communications, compared to four for source-cut partitioning.

C. Processing-in-Memory (PIM) Architecture

As shown in Fig. 4(a), a typical PIM system consists of multiple memory stacks connected via external links, with a dragonfly-based network widely adopted for its simple topology and high performance [8]. This network topology includes multiple interconnected groups, with each group fully connected internally and fully interconnected with other groups. Each stack is managed using a memory controller (MC). Fig. 4(b) illustrates a PIM architecture enabled by 3D-stacking technology. Multiple DRAM dies are stacked on a base die and interconnected by through-silicon vias (TSVs). Each DRAM die is organized into channels, with each channel comprising bank groups and each bank group comprising individual banks. To reduce off-chip data movement, many PIM systems [18] integrate compute logic directly on the base die. The emerging PIM architectures [3], [15], [19] further integrate compute and control logic near memory banks, forming a PIM unit, as depicted in Fig. 4(c).

D. Motivation: Rethinking Graph Partitioning for PIM

Graph partitioning for PIM architecture requires careful design to reduce data movement. Since data movement is

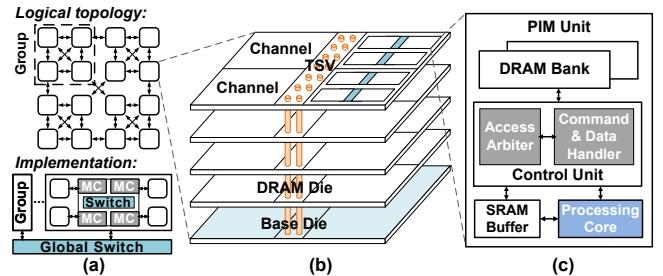


Fig. 4: A PIM system. (a) Dragonfly-based topology. (b) 3D-stacked PIM architecture. (c) PIM-enabled bank group.

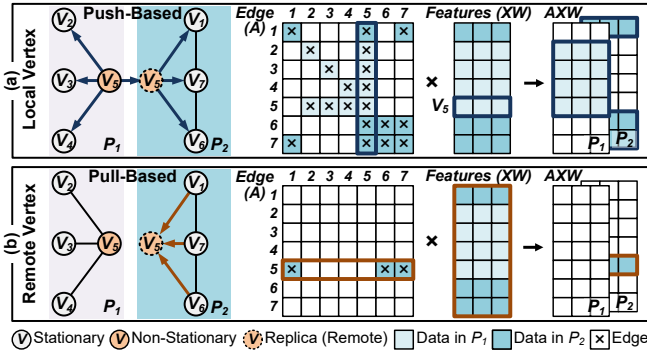


Fig. 7: Hybrid dataflow.

on vertex degrees, and it has a time complexity of $O(|E|)$. The vertex clustering calculates the maximum modularity increment by traversing the neighbors of each vertex, also resulting in a time complexity of $O(|E|)$.

B. Minimizing Non-Stationary Data Movement

Non-stationary vertices are often required by multiple PIM units, incurring repeated data transfers. To minimize data movement, Anchor introduces a two-step optimization with hardware support. A hybrid dataflow ensures each non-stationary vertex is distributed only once, and optimized communication reduces redundant movement during distribution.

1) Hybrid Dataflow with Hardware Support:

Hybrid Dataflow. Anchor employs a hybrid dataflow for aggregation, combining push-based and pull-based approaches. The push-based dataflow broadcasts features from local vertices to their neighbors along outgoing edges, as shown in Fig. 7(a), where vertex V_5 broadcasts to V_1, V_6 , and V_7 in P_2 and to V_2, V_3 , and V_4 in P_1 . This ensures that features of non-stationary vertex V_5 are distributed once, minimizing redundant accesses and transfers. The push-based dataflow processes edges in column order, and thus a compressed sparse row (CSR) format is used for edges of local vertices. Processing remote vertices with a push-based dataflow can exhaust memory due to the need to store all intermediates during aggregation. Therefore, the pull-based dataflow is adopted to handle remote vertices. As shown in Fig. 7(b), remote vertex V_5 in P_2 pulls features from neighbors, V_1, V_6 , and V_7 , along incoming edges. The pull-based dataflow processes edges in row order using a compressed sparse column (CSC) format.

Hardware Support. As shown in Fig. 8, the PIM unit is redesigned with an aggregation unit, an aggregation queue, and a vertex locator to support the proposed hybrid dataflow. The aggregation unit includes MAC arrays and dedicated SRAM buffers for aggregation. The vertex locator manages the allocation of space and the search for vertices addresses, and the aggregation queue stores vertex IDs prepared for aggregation. Similar to previous works [19], the access arbiter manages memory access within and outside the PIM unit, and the handlers process external instructions to enable cross PIM unit communication.

Anchor organizes the bank space into three distinct zones: edge data zone, local vertex zone, and remote vertex zone,

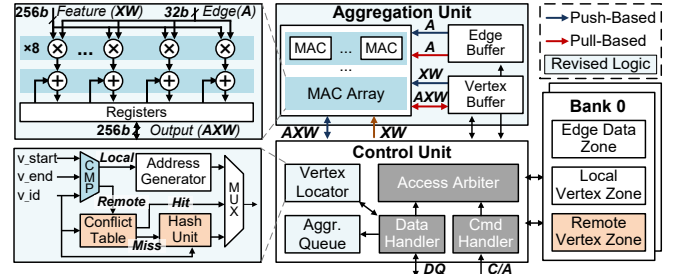


Fig. 8: Architecture design of the PIM unit.

to support the hybrid dataflow. The edge data zone stores the weights of edges assigned to the PIM unit. The local vertex zone holds the features of the vertices allocated to the PIM unit. The remote vertex zone temporarily caches features of non-stationary vertices from other PIM units, utilizing huge DRAM space to prevent blocking of remote vertex delivery. Anchor employs a hash unit and a conflict table to manage the space allocation of the remote vertex zone.

The aggregation unit supports both push-based and pull-based dataflows, as shown in Fig. 8. Local vertices are aggregated using the push-based dataflow. First, a portion of edge weights (A) is read from the edge data zone into the edge buffer. Second, IDs of prepared vertices are fetched from the head of the aggregation queue, and their features (XW) are retrieved from the banks to the vertex buffer. Third, the MAC arrays then perform matrix multiplication with edges and vertex features, accumulating the resulting output (AXW) back into the local vertex zone. Remote vertices are aggregated using the pull-based dataflow, following a similar process.

Anchor defines three types of extended instructions for GNNs, using standard DRAM commands with reserved DRAM rows and columns [15], [19]. The Sync instruction inserts vertex IDs into the aggregation queue, Replic stores remote vertex features to banks, and Merge fetches aggregated replicas from banks to the base die. The data and command handlers decode these instructions.

2) Optimized Communication with Hardware Support:

Optimized Communication. The proposed customized dataflow enables simultaneous processing of each non-stationary vertex, facilitating communication optimization. As shown in Fig. 9(a), Anchor uses a broadcast mechanism to optimize replicating operations during the aggregation phase. Local broadcast is used to reduce intra-stack data movement, where vertices updated after the combination operation are directly broadcast from the base die to PIM units via TSVs. To optimize inter-stack communication, Anchor utilizes the hierarchical structure of the dragonfly network [26], employing a tree-based global broadcast to minimize redundant data movement through critical paths. As illustrated in Fig. 9(a), the repeated access paths ($S00, S01$), ($S00, S02$), ($S02, S08$), and ($S01, S04$) are reduced from two accesses each to a single access. Similarly, a tree-based reduction is adopted for the merging phase, as depicted in Fig. 9(b). This phase conducts both local and global reduction. Local reduction pulls local replicas to the base die for merging, while global reduction

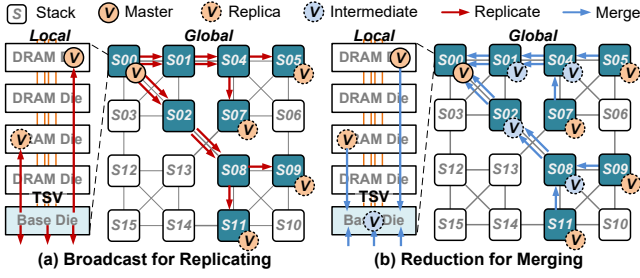


Fig. 9: Communication optimization mechanisms.

merges replicas from child nodes in the reduction tree.

Hardware support. Fig. 10 illustrates the architecture design of the base die logic. A combination and merging unit (MCU) is added for the combination and merging phase. An internal communication (IC), an external communication (EC) controller, and a synchronization unit are designed to support the proposed communication optimization. The IC controller generates commands and instructions to interact with PIM units, while the EC controller packets and delivers data to other stacks. To manage the delivery of vertices, a sliding window approach is adopted [28]. The synchronization unit distributes vertex IDs prepared for aggregation to local PIM units.

During the combination phase, the MCU uses a pull-based dataflow to perform matrix multiplication similar to the aggregation phase. After the combination phase, each updated vertex is delivered by the IC and EC controllers to different PIM units. Stationary vertices are directly returned to their allocated PIM units via the IC controller using the `Replic` instruction. Non-stationary vertices are broadcast by both the IC and EC controllers. Vertices that complete `Replic` instructions are sent to the synchronization unit, which then broadcasts `Sync` instructions to the PIM units for further aggregation. During the merging phase, the IC controller issues `Merge` instructions to PIM units, pulling aggregated replicas to the base die for local reduction. Once a vertex completes local reduction, its intermediate result (*Mid*) is delivered by the EC controller for global reduction. Finally, vertices that have completed merging are returned to their allocated PIM units.

IV. EVALUATION

A. Experimental setup

Models and Datasets. To evaluate our scheme, we employ a set of representative GNN models, including GCN [14], GIN [23], and GraphSage [11]. Each GNN model consists of

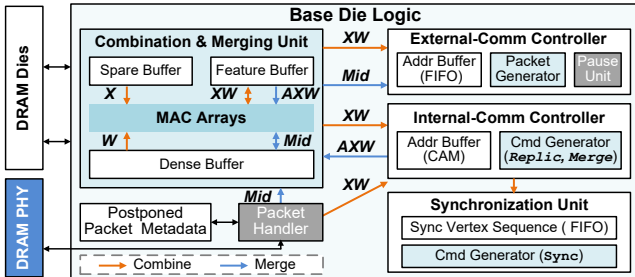


Fig. 10: Architecture design of the base die logic.

TABLE I: Graph datasets.

Dataset	# of vertices	# of edges	# of features	# of classes
Amazon (AM)	1,569,960	264,339,468	200	107
Cora (CR)	2,708	10,556	1,433	7
Flickr (FL)	89,250	989,006	500	7
Pubmed (PM)	19,717	88,648	500	3
Reddit (RD)	232,965	114,615,892	602	41
Yelp (YL)	716,847	13,954,819	300	100

TABLE II: System configurations and power-area models.

Configuration	
PIM System	16 stacks $\times$ 16 channels $\times$ (1 bank group + 3 PIM bank groups) $\times$ 4 banks, 8GB per stack, 48 PIM units per stack, 16GB/s intra-stack bandwidth per channel, 64GB/s inter-stack bandwidth per bidirectional link.
PIM unit	4 $\times$ 8 MACs @ 125MHz, 1KB conflict table, 2KB edge buffer, 2KB vertex buffer, Power=42.66mW, Area=0.18mm ² (4.9 % of a bank group)
Base Die Logic	32 $\times$ 16 MACs @ 1MHz, 64KB sparse buffer, 32KB feature buffer, 1MB dense buffer, 8KB address buffer (FIFO), 8KB address table (CAM), Power=2719.68mW, Area=9.48mm ² (10.6% of the base die)

two layers with a hidden size of 64. As listed in Table I, we utilize six commonly used graph datasets [2] for evaluation.

Implementation and Configuration. We modify and extend DRAMSim3 [17], a cycle-accurate simulator, to support in-memory operations for GNNs. The area and power consumption of the processing units are obtained by implementing Anchor in Verilog and synthesizing the RTL using the Synopsys Design Compiler with a 32nm standard library. We employ CACTI [1] to estimate the SRAM buffer, interconnect components, and the DRAM stack. Detailed configurations, power models, and area models are listed in Table II.

Baselines. At the hardware level, we compare Anchor with state-of-the-art architectures, including HyGCN [24], GROW [12], GCIM [6], and Lift [7]. HyGCN and GROW are typical GNN accelerators, whereas GCIM and Lift are near-bank PIM architectures for GNNs. At the software level, we compare the proposed Mastav with graph partitioning methods, including METIS [13], DBH [21], and Lift [7].

B. Overall Performance

1) *Speedup*: Fig. 11 illustrates the performance speedup across various benchmarks, with Anchor outperforming all baselines. Specifically, Anchor achieves average speedups of 13.01 $\times$, 3.57 $\times$, 10.12 $\times$, and 3.15 $\times$ over HyGCN, GROW, GCIM, and Lift, respectively. This performance gain is primarily driven by PIM architectures, which offer substantial processing bandwidth and minimize data movement. Additionally, Anchor further reduces data movement across PIM

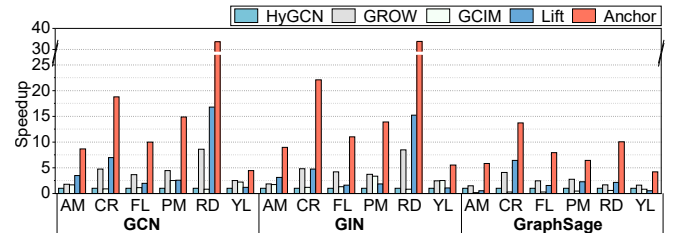


Fig. 11: Performance speedup.

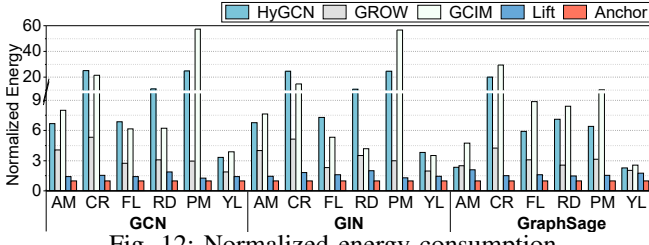


Fig. 12: Normalized energy consumption.

units by efficiently leveraging stationary data, enabling it to outperform other PIM architectures, including GCIM and Lift.

2) *Energy Consumption*: Fig. 12 presents the normalized energy consumption across various benchmarks, demonstrating that Anchor achieves the lowest average energy consumption. Anchor achieves energy savings of $11.10\times$, $3.19\times$, $14.40\times$, and $1.58\times$ compared to HyGCN, GROW, GCIM, and Lift, respectively. This reduction is attributed to the hybrid dataflow, which minimizes unnecessary memory accesses, and optimized communication mechanisms that reduce redundant data movement.

C. Comparison of Graph Partitioning Algorithms

1) *Performance and Scalability*: Fig. 13 shows the speedups for different graph partitioning algorithms as the number of stacks increases. It can be observed that Mastav achieves the highest speedup among the baseline algorithms. As the stack number increases from 4 to 16 and 64, Mastav improves performance by an average of $3.22\times$ and $7.16\times$.

2) *Data Movement*: Fig. 14(a) presents the data movement for different graph partitioning algorithms. Compared to Lift and DBH, Mastav reduces data movement by 40.55% and 38.44%, respectively. For the AM, CR, and PM graphs, METIS incurs less data movement than Mastav by prioritizing the minimization of cross-partition edges, often at the cost of workload balance. In contrast, Mastav effectively reduces data movement while maintaining balanced workloads.

3) *Energy Breakdown*: Fig. 14 (b) illustrates the normalized energy breakdown for various graph partitioning algorithms, where ‘M’, ‘L’, ‘D’, and ‘A’ denote METIS, Lift, DBH, and Mastav, respectively. Mastav achieves the lowest energy consumption, reducing energy usage by 28.15%, 26.30%, and 35.07% compared to METIS, Lift, and DBH, respectively. This reduction is attributed to Mastav’s ability to minimize data movement, thereby lowering energy consumption.

4) *Stationary Vertex Ratio*: Fig. 15 shows the ratio of stationary vertices for different graph partitioning algorithms as the number of stacks increases. Both METIS [13] and Lift [7], which utilize source-cut edge partitioning, experience

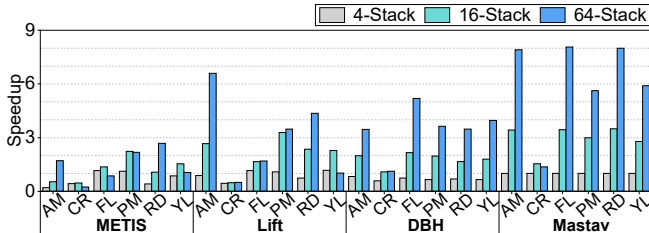


Fig. 13: Performance and scalability.

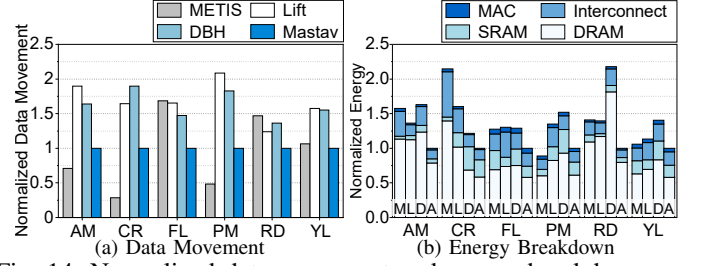


Fig. 14: Normalized data movement and energy breakdown.

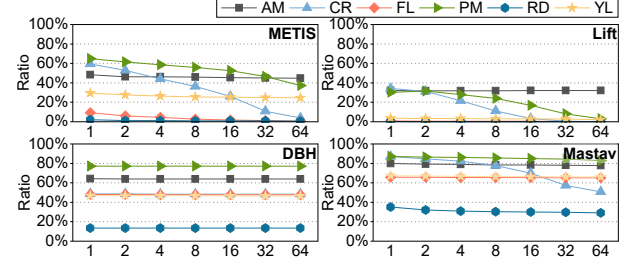


Fig. 15: Ratios of stationary vertices for different graph partitioning algorithms as the number of stacks increases.

a sharp decline in stationary vertex ratios as the stack count increases, dropping from 35.6% to 18.7% for METIS and from 16.9% to 6.3% for Lift. In contrast, DBH and Mastav, employing mixed-cut partitioning, maintain a stable stationary vertex ratio as the stack count increases. Compared to DBH, Mastav leverages clustering to optimize locality, achieving the highest stationary ratios, ranging from 62.0% to 70.5%.

V. CONCLUSION

This paper presents Anchor, a scalable PIM architecture for GNNs through a hardware-software co-design approach. Following the Max-Min principle, we focus on maximizing stationary data and minimizing the movement overhead of non-stationary data. A graph partitioning algorithm, Mastav, is proposed to maintain data in place. Furthermore, an optimized dataflow and a communication mechanism are introduced to reduce memory access and data movement. Experimental results demonstrated that Anchor significantly reduces latency and data movement compared to existing schemes. In the future, we plan to explore more architectural characteristics of PIM architecture to handle dynamic and more complex graph workloads.

ACKNOWLEDGEMENTS

The work described in this paper is supported in part by NSFC (U23B2040), in part by National Key Research and Development Program of China (2024YFB4505702), in part by Shenzhen Science and Technology Program (RCJC20221008092725019), in part by Guangdong Province Key Laboratory of Popular High Performance Computers (2017B030314073), in part by Guangdong Province Engineering Center of China-made High Performance Data Computing System and Shenzhen Key Laboratory of Service Computing and Applications. Yi Wang is the corresponding author.

REFERENCES

- [1] “CACTI,” <https://www.hpl.hp.com/research/cacti>, 2024.
- [2] “Pytorch Geometric,” https://github.com/pyg-team/pytorch_geometric, 2024.
- [3] “UPMEM,” <https://www.upmem.com/>, 2024.
- [4] J. Arai, H. Shiokawa, T. Yamamuro, M. Onizuka, and S. Iwamura, “Rabbit Order: Just-in-Time Parallel Reordering for Fast Graph Analysis,” in *International Parallel and Distributed Processing Symposium (IPDPS)*, 2016, pp. 22–31.
- [5] C. Chen, K. Li, Y. Li, and X. Zou, “ReGNN: A Redundancy-Eliminated Graph Neural Networks Accelerator,” in *International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 429–443.
- [6] J. Chen, Y. Lin, K. Sun, J. Chen, C. Ma, R. Mao, and Y. Wang, “GCIM: Toward Efficient Processing of Graph Convolutional Networks in 3D-Stacked Memory,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, pp. 3579–3590, 2022.
- [7] J. Chen, Z. Zhong, K. Sun, C. Ma, R. Mao, and Y. Wang, “Lift: Exploiting Hybrid Stacked Memory for Energy-Efficient Processing of Graph Convolutional Networks,” in *Design Automation Conference (DAC)*, 2023, pp. 1–6.
- [8] A. Fatima, S. Liu, K. Seemakhupt, R. Ausavarungnirun, and S. Khan, “vPIM: Efficient Virtual Address Translation for Scalable Processing-in-Memory Architectures,” in *Design Automation Conference (DAC)*, 2023, pp. 1–6.
- [9] T. Geng, A. Li, R. Shi, C. Wu, T. Wang, Y. Li, P. Haghi, A. Tumeo, S. Che, S. Reinhardt, and M. C. Herbordt, “AWB-GCN: A Graph Convolutional Network Accelerator with Runtime Workload Rebalancing,” in *International Symposium on Microarchitecture (MICRO)*, 2020, pp. 922–936.
- [10] T. Geng, C. Wu, Y. Zhang, C. Tan, C. Xie, H. You, M. Herbordt, Y. Lin, and A. Li, “I-GCN: A Graph Convolutional Network Accelerator with Runtime Locality Enhancement through Islandization,” in *International Symposium on Microarchitecture (MICRO)*, 2021, pp. 1051–1063.
- [11] W. L. Hamilton, Z. Ying, and J. Leskovec, “Inductive Representation Learning on Large Graphs,” in *Advances in Neural Information Processing Systems (NIPS)*, 2017, pp. 1024–1034.
- [12] R. Hwang, M. Kang, J. Lee, D. Kam, Y. Lee, and M. Rhu, “GROW: A Row-Stationary Sparse-Dense GEMM Accelerator for Memory-Efficient Graph Convolutional Neural Networks,” in *International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 42–55.
- [13] G. Karypis and V. Kumar, “A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs,” *SIAM Journal on Scientific Computing*, pp. 359–392, 1998.
- [14] T. N. Kipf and M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” in *International Conference on Learning Representations (ICLR)*, 2017, pp. 1–14.
- [15] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, “Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product,” in *International Symposium on Computer Architecture (ISCA)*, 2021, pp. 43–56.
- [16] J. Li, A. Louri, A. Karanth, and R. Bunesu, “GCNAX: A Flexible and Energy-Efficient Accelerator for Graph Convolutional Neural Networks,” in *International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 775–788.
- [17] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, “DRAMsim3: A Cycle-Accurate, Thermal-Capable DRAM Simulator,” *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.
- [18] B. Tian, Q. Chen, and M. Gao, “ABNDP: Co-optimizing Data Access and Load Balance in Near-Data Processing,” in *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023, pp. 3–17.
- [19] B. Tian, Y. Li, L. Jiang, S. Cai, and M. Gao, “NDPBridge: Enabling Cross-Bank Coordination in Near-DRAM-Bank Processing Architectures,” in *International Symposium on Computer Architecture (ISCA)*, 2024, pp. 1–16.
- [20] H. Wang, S. Zhang, X. Fan, Z. Yang, and M. Zhang, “NDPGNN: A Near-Data Processing Architecture for GNN Training and Inference Acceleration,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, pp. 3997–4008, 2024.
- [21] C. Xie, L. Yan, W.-J. Li, and Z. Zhang, “Distributed Power-law Graph Computing: Theoretical and Empirical Analysis,” in *Advances in Neural Information Processing Systems (NIPS)*, 2014, pp. 1–9.
- [22] T. Xie, T. Zhao, Z. Zhu, X. Ning, B. Li, G. Dai, H. Yang, and Y. Wang, “DyPIM: Dynamic-Inference-Enabled Processing-In-Memory Accelerator,” in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2024, pp. 1–6.
- [23] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How Powerful are Graph Neural Networks?” in *International Conference on Learning Representations (ICLR)*, 2019, pp. 1–17.
- [24] M. Yan, L. Deng, X. Hu, L. Liang, Y. Feng, X. Ye, Z. Zhang, D. Fan, and Y. Xie, “HyGCN: A GCN Accelerator with Hybrid Architecture,” in *International Symposium on High-Performance Computer Architecture (HPCA)*, 2020, pp. 15–29.
- [25] H. Yu, Y. Zhang, L. He, D. He, Q. Li, J. Zhao, X. Liao, H. Jin, L. Gu, and H. Liu, “CDA-GNN: A Chain-driven Accelerator for Efficient Asynchronous Graph Neural Network,” in *Design Automation Conference (DAC)*, 2024, pp. 1–6.
- [26] M. Zhang, Y. Zhuo, C. Wang, M. Gao, Y. Wu, K. Chen, C. Kozyrakis, and X. Qian, “GraphP: Reducing Communication for PIM-Based Graph Processing with Efficient Data Partition,” in *International Symposium on High Performance Computer Architecture (HPCA)*, 2018, pp. 544–557.
- [27] C. Zhao, R. D. Chamberlain, and X. Zhang, “SuperCut: Communication-Aware Partitioning for Near-Memory Graph Processing,” in *International Conference on Computing Frontiers (CF)*, 2023, pp. 42–51.
- [28] Z. Zhou, C. Li, X. Wei, X. Wang, and G. Sun, “GNNear: Accelerating Full-Batch Training of Graph Neural Networks with Near-Memory Processing,” in *International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2023, pp. 54–68.
- [29] Z. Zhou, B. Shi, Z. Zhang, Y. Guan, G. Sun, and G. Luo, “BlockGNN: Towards Efficient GNN Acceleration Using Block-Circulant Weight Matrices,” in *Design Automation Conference (DAC)*, 2021, pp. 1009–1014.
- [30] Z. Zhu, F. Li, G. Li, Z. Liu, Z. Mo, Q. Hu, X. Liang, and J. Cheng, “MEGA: A Memory-Efficient GNN Accelerator Exploiting Degree-Aware Mixed-Precision Quantization,” in *International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 124–138.